

SOCCORSO WEB

Documentazione del progetto

SoccorsoWeb Services - REST API e client JavaScript

**Corso di Sviluppo Web Avanzato
A.A. 2025/2026**

Docente: Prof. Giuseppe Della Penna

Luca Evangelista
Matricola 252635
luca.evangelista1@student.univaq.it

Versione 1.0 - 15 luglio 2026

Sintesi

Il progetto realizza un servizio RESTful per la gestione di richieste di soccorso, operatori, squadre, missioni, mezzi e materiali. Il server è sviluppato in Java 21 con JAX-RS/Jersey e persistenza JDBC su MySQL; il client di test è composto da pagine HTML, CSS e JavaScript che invocano le API tramite Fetch. L'autenticazione usa JWT firmati HMAC, trasmessi principalmente tramite cookie o header Bearer, con controllo dei ruoli e revoca mediante blacklist su database.

La versione finale registra 10 classi risorsa, espone 34 operazioni REST, contiene 15 pagine client e un database relazionale di 16 tabelle. La specifica OpenAPI 3.0.3 completa è riportata nell'Appendice B ed è fornita anche come file YAML separato.

Indice dei contenuti

- 1. Obiettivi e fonti del documento
- 2. Conformità alla specifica
- 3. Dipendenze software e ambiente
- 4. Architettura e organizzazione del codice
- 5. Modello dei dati
- 6. Autenticazione, autorizzazione e JWT
- 7. Client JavaScript e navigazione
- 8. Funzionalità realizzate
- 9. Convenzioni dell'API
- 10. Catalogo completo degli endpoint
- 11. Funzionalità extra
- 12. Installazione, configurazione e collaudo
- Appendice A. Schemi JSON
- Appendice B. Specifica OpenAPI 3.0.3 completa
- Appendice C. Contributo individuale

1. Obiettivi e fonti del documento

Questa relazione documenta la versione finale del progetto Soccorso Web SWA. La struttura segue i requisiti della specifica "SoccorsoWeb Services" e usa come riferimento la documentazione del precedente progetto Web Engineering.

- codice Java, risorse JAX-RS, filtri di sicurezza, DAO e modelli contenuti nel progetto finale;
- pom.xml per dipendenze e versioni;
- script SQL per schema, vincoli e dati iniziali;
- pagine HTML, fogli CSS e script JavaScript per la gestione lato client;

Ambito

La relazione descrive il comportamento effettivamente ricavabile dal codice. Le classi Servlet/FreeMarker provenienti dal progetto precedente sono ancora presenti nell'archivio, ma l'API attiva è quella registrata esplicitamente in RestApplication.

2. Conformità alla specifica

N.	Funzionalità	Stato	Implementazione
1	Login/logout	Realizzata	POST /auth/login, DELETE /auth/logout; aggiunto GET /auth/refresh.
2	Inserimento richiesta	Realizzata	POST /richieste, pubblico, UUID generato lato server.
3	Convalida richiesta	Realizzata	GET /convalida?uuid=...; transizione pendente -> attiva.
4	Lista paginata e filtrata	Realizzata	GET /richieste/list?page=&size=&stato=.
5	Operatori liberi	Realizzata	GET /operatori/list?stato=libero.
6	Creazione missione	Realizzata	POST /missioni con squadra, mezzi e materiali.
7	Chiusura missione	Realizzata	PUT /missioni/{id}/terminazione.
8	Annullamento richiesta	Realizzata	PUT /richieste/{id}/rifiutata.
9	Dettaglio missione	Realizzata	Missione + sotto-risorse squadra, mezzi e materiali.
10	Dettaglio richiesta	Realizzata	GET /richieste/{id}.
11	Dettaglio operatore	Realizzata	GET /operatori/{id}, patenti, abilità e missione attiva.
12	Missioni operatore	Realizzata	GET /operatori/{id}/storico.

Il client richiesto dalla traccia copre esplicitamente login/logout, invio richiesta, lista paginata/filtrata delle richieste e lista degli operatori liberi. La versione finale aggiunge interfacce per missioni, dettagli, mezzi, materiali e gestione delle competenze.

3. Dipendenze software e ambiente

3.1 Dipendenze server

Componente	Versione	Ruolo
Java	21	Linguaggio e livello di compilazione configurato dal maven-compiler-plugin.
Jakarta Servlet API	6.0.0	API del container; dipendenza provided.
Jersey container servlet	3.1.11	Implementazione JAX-RS e integrazione con il container Servlet.
Jersey HK2	3.1.11	Dependency injection usata dal runtime Jersey.
MySQL Connector/J	8.3.0	Driver JDBC per MySQL.
Jackson Jakarta-RS JSON provider	2.22.0	Serializzazione/deserializzazione JSON per JAX-RS.
Jackson datatype JSR-310	2.20.0	Supporto LocalDate e LocalDateTime.
Jackson Databind tools.jackson	3.2.0	Presente nel POM e usato dalla Servlet didattica MezziAPI.
JWT API / impl / jackson	0.13.0	Creazione, firma e verifica dei JWT.
Swagger JAX-RS2 Jakarta	2.2.52	Esposizione della descrizione OpenAPI.
Swagger UI WebJar	5.31.0	Interfaccia interattiva per consultare e provare gli endpoint.
FreeMarker	2.3.34	Residuo del progetto precedente; non necessario al client statico finale.

3.2 Build e runtime

- Packaging Maven WAR, artifactId soccorso_Web_SWA, versione 0.0.1-SNAPSHOT.
- maven-compiler-plugin 3.13.0 con release 21 e maven-war-plugin 3.2.3.
- Container compatibile con Jakarta Servlet 6, ad esempio Apache Tomcat 10.1.
- MySQL raggiungibile su localhost:3306, database soccorso_web.

3.3 Dipendenze client

- HTML5, CSS3 e JavaScript con Fetch API, Promise e DOM API.
- Bootstrap 5.0.2/5.3.3 richiamato da CDN nelle pagine più recenti; asset SB Admin 2 e Bootstrap 4 conservati nel progetto.
- Font Awesome, jQuery, Chart.js e DataTables presenti tra gli asset ereditati; il client REST principale usa soprattutto JavaScript nativo.
- Swagger UI disponibile in swagger-ui.html e configurata su /soccorso_Web_SWA/rest/openapi.json.

4. Architettura e organizzazione del codice

L'architettura è organizzata a livelli. Il client statico esegue chiamate HTTP verso le risorse JAX-RS; il filtro di autenticazione valida il JWT e costruisce il SecurityContext; le risorse delegano la persistenza ai DAO, che usano JDBC e PreparedStatement per interagire con MySQL.

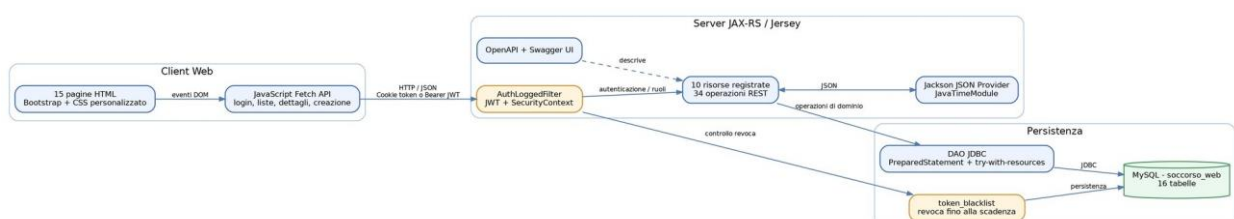


Figura 1 - Architettura logica della versione REST del progetto.

4.1 Package Java

Package	Responsabilità
eva.luca.soccorso_Web.base	RestApplication: registra risorse, filtro, provider Jackson e OpenApiResource.
...resources	Risorse JAX-RS per autenticazione, richieste, missioni, operatori, mezzi, materiali, cataloghi e squadre.
...security	AuthLoggedFilter e classi di sicurezza legacy.
...utility	Annotazione @Logged, gestione JWT, helper di autenticazione e simulazione email.
...data	DAO JDBC e interfacce generiche IDaoRead/IDaoWrite.
...data.db	Configurazione e factory delle connessioni JDBC.
...models	POJO serializzati in JSON, risposte di errore/successo e risposta paginata.
...controllers	Servlet e viste FreeMarker del progetto precedente, non centrali nel client REST finale.

4.2 Risorse effettivamente pubblicate

AuthRes, ConvalidaRes, RichiesteRes, MissioniRes, OperatoriRes, MezziRes, MaterialiRes, PatentiRes, CompetenzeRes, SquadreRes.

Codice non registrato

AdminRes non viene aggiunta a RestApplication e quindi non fa parte dell'API documentata. MezziAPI è una Servlet didattica separata su /api/mezzi/* e non appartiene al namespace JAX-RS /rest.

5. Modello dei dati

Lo script SQL crea 16 tabelle. Le entità principali sono operatori, amministratori, richieste, missioni, squadre, mezzi e materiali; le relazioni multi-a-multi sono normalizzate in tabelle ponte. La blacklist JWT è persistita separatamente.

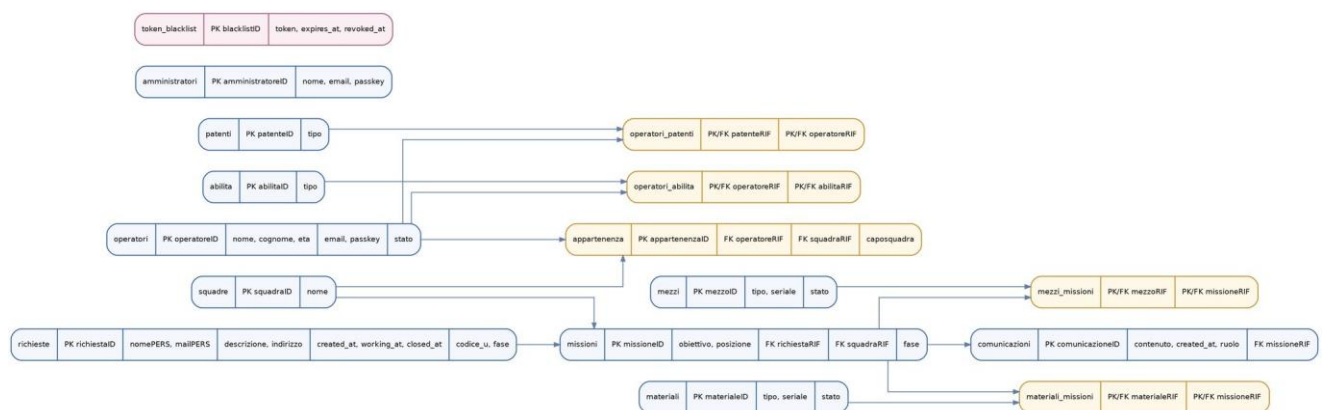


Figura 2 - Schema relazionale semplificato ricostruito dallo script Esame.sql.

5.1 Vincoli e stati

- richieste.fase: pendente, attiva, in esecuzione, terminata, rifiutata;
- missioni.fase: attiva o terminata;
- operatori.stato, mezzi.stato e materiali.stato: libero o occupato;
- email univoca per operatori e amministratori; seriale univoco per mezzi e materiali;
- chiavi composte nelle tabelle ponte per impedire associazioni duplicate;
- cancellazioni in cascata sulle principali foreign key.

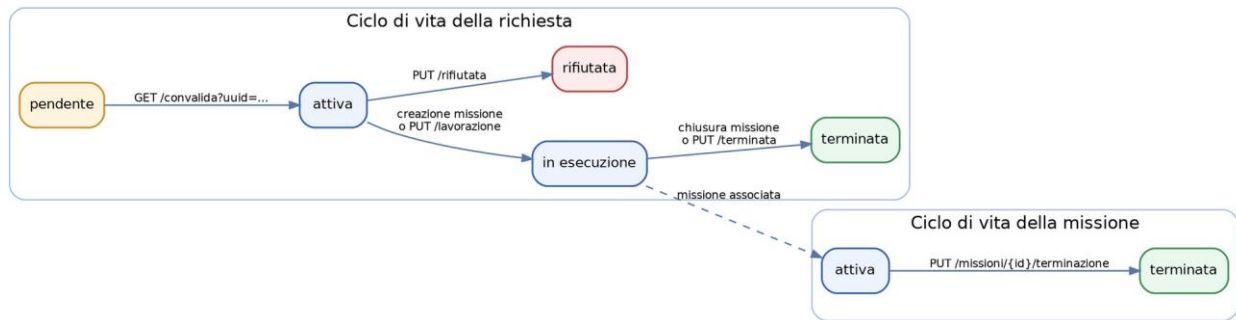


Figura 3 - Stati applicativi e transizioni esposte dall'API.

5.2 Coordinamento delle risorse operative

Alla creazione di una missione vengono impostati come occupati i membri della squadra, i mezzi e i materiali associati; la richiesta passa in esecuzione. Alla terminazione, le risorse tornano libere e richiesta e missione vengono chiuse. Le query di SquadraDao selezionano squadre non assegnate a missioni attive e senza membri occupati.

6. Autenticazione, autorizzazione e JWT

Il login riceve credenziali form-urlencoded. AuthDao verifica prima la tabella amministratori e poi operatori; il ruolo risultante viene inserito nel claim ruolo. Il token ha durata di 15 minuti ed è firmato HMAC-SHA256.

6.1 Contenuto del token

```

{
  "sub": "roberto.mancini@mail.com",
  "ruolo": "admin",
  "iss": "http://localhost:8080/soccorso_Web_SWA/rest/auth/login",
  "iat": 1784100000,
  "exp": 1784100900
}

```

6.2 Trasmissione e filtro

1. Header Authorization: Bearer <token> (priorità più alta).
2. Cookie token, impostato dal login con path /.
3. Parametro query token, mantenuto come modalità didattica di fallback.

AuthLoggedFilter è un name-bound filter applicato alle classi o ai metodi annotati con @Logged. Dopo la validazione inserisce token, email e ruolo come proprietà della request e sostituisce il SecurityContext con un'implementazione che supporta isUserRole().

6.3 Matrice di autorizzazione

Livello	Operazioni principali
Pubblico	POST /richieste; GET /convalida; POST /auth/login
Qualsiasi autenticato	logout/refresh; dettaglio operatore e sotto-risorse; associazione patenti/abilità; cataloghi; squadre; PUT richiesta terminata
Solo admin	liste e dettagli richieste; creazione/rifiuto/avvio richiesta; missioni; lista/creazione/dettaglio operatori; mezzi; materiali

6.4 Revoca

Il logout verifica il JWT, legge la scadenza e inserisce token ed expires_at nella tabella token_blacklist. Ogni validazione controlla che il token non sia presente e ancora non scaduto nella blacklist.

7. Client JavaScript e navigazione

Il client è composto da pagine HTML autonome e script JavaScript associati. Le chiamate protette usano credentials: "same-origin" o "include" affinché il cookie JWT sia inviato al server. Gli errori JSON vengono letti anche per risposte e mostrati nella pagina.

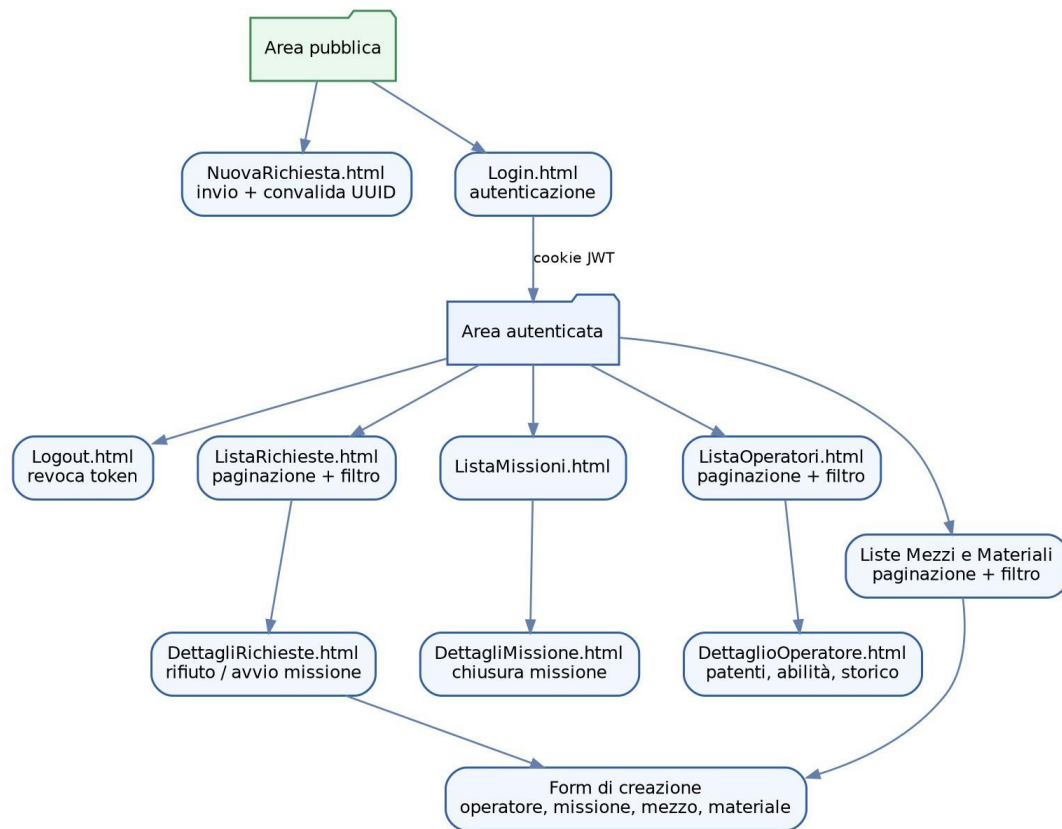


Figura 4 - Navigazione del client di test e principali gruppi di pagine.

Componente	Funzione
NuovaRichiesta.html / NuovaRichiesta.js	Invio pubblico e convalida tramite uuid nella query string.
Login.html / Login.js	Login form-urlencoded, cookie e token in sessionStorage.
Logout.html / Logout.js	Conferma logout, DELETE autenticata e pulizia storage.
ListaRichieste.html / RqstScript.js	Paginazione server-side, filtro di fase e apertura dettagli.
DettagliRichieste.html	Visualizzazione, rifiuto e avvio della creazione missione.
ListaMissioni.html / DettagliMissione.html	Lista, dettagli, squadra e terminazione.
ListaOperatori.html / DettaglioOperatore.html	Paginazione, filtro, patenti, abilità, missione attiva e storico.
Liste/Nuovi Mezzi e Materiali	Paginazione, filtro e inserimento.
NuovoOperatore.html / NuovaMissione.html	Creazione delle entità e selezione delle risorse disponibili.

8. Funzionalità realizzate

8.1 Richieste di soccorso

L'utente pubblico inserisce nome, email, indirizzo e descrizione. Il server impedisce un secondo invio dalla stessa email entro due minuti, genera un UUID e salva la richiesta come pendente. La simulazione email produce un link alla pagina pubblica con il parametro uuid; la convalida porta la richiesta ad attiva. L'amministratore può consultare, filtrare, rifiutare o mettere in lavorazione la richiesta.

8.2 Missioni

La missione nasce da una richiesta e riceve obiettivo, posizione, squadra, mezzi e materiali. Il server impedisce di creare più missioni per la stessa richiesta, inserisce le associazioni nelle tabelle ponte, occupa le risorse e porta la richiesta in esecuzione. La chiusura libera tutte le risorse e termina missione e richiesta.

8.3 Operatori e competenze

La lista operatori è paginata e filtrabile. Il dettaglio espone i dati anagrafici, l'eventuale missione attiva, patenti, abilità e storico missioni. Le credenziali sono accettate in creazione, ma passkey è write-only e non viene inclusa nel JSON di risposta.

8.4 Mezzi e materiali

Entrambe le risorse supportano lista paginata, filtro per disponibilità, dettaglio e inserimento. Lo stato iniziale è libero e viene aggiornato automaticamente dalle operazioni sulle missioni.

8.5 Squadre

L'endpoint restituisce solo squadre non operative: non associate a una missione attiva e senza operatori occupati. Questa scelta evita assegnazioni concorrenti.

8.6 Cataloghi

Patenti e abilità sono entità separate e riutilizzabili. Gli endpoint di catalogo alimentano le select del dettaglio operatore, mentre le associazioni sono salvate in tabelle multi-a-molti.

9. Convenzioni dell'API

Aspetto	Convenzione
Base URL	/soccorso_Web_SWA/rest
Formato principale	application/json; login e token usano text/plain / form-urlencoded.
Identificativi	Interi positivi nel path, con regex JAX-RS {id:[0-9]+}.
Paginazione	page da 1; size da 1 a 100; risposta PaginatedResponse.
Date	ISO-8601: yyyy-MM-dd per LocalDate e yyyy-MM-ddTHH:mm:ss per LocalDateTime.
Errori	Oggetto JSON {"message":"..."}, gestite con errorMessage.
Autenticazione	JWT Bearer o cookie token; 401 per token non valido, 403 per ruolo insufficiente.
Collection-item	Collezioni e item separati; relazioni rappresentate con path nidificati, ad esempio /missioni/{id}/mezzi.

9.1 Esempio di risposta paginata

```
{
  "content": [
    {
      "id": 3,
      "nomePersona": "Anna Bianchi",
      "mailPersona": "anna@mail.com",
      "descrizione": "Allagamento della cantina dopo un temporale",
      "indirizzo": "Via Milano 22, L'Aquila",
      "createdAt": "2026-07-15T08:42:10",
      "workingAt": null,
      "closedAt": null,
      "fase": "attiva",
      "uuid": "8b0a6a1d-0a24-42f6-a340-11e9e840a120"
    }
  ],
  "page": 1,
}
```



```

"size": 10,
"totalElements": 25,
"totalPages": 3,
"first": true,
"last": false
}

```

9.2 Esempio di errore

```

{
  "message": "Stato non valido. Valori consentiti: attiva, in esecuzione, terminata, rifiutata"
}

```

10. Catalogo completo degli endpoint

La tabella seguente riepiloga le 34 operazioni effettivamente registrate. Le schede successive specificano parametri, payload, esempi e codici di stato.

Verbo	URL relativa	Accesso	Operazione
POST	/auth/login	Pubblico	Login
DELETE	/auth/logout	Autenticato	Logout
GET	/auth/refresh	Autenticato	Refresh token
GET	/convalida	Pubblico	Convalida richiesta
GET	/richieste/list	Admin	Lista richieste paginata
GET	/richieste/{id}	Admin	Dettaglio richiesta
POST	/richieste	Pubblico	Inserimento richiesta
PUT	/richieste/{id}/rifiutata	Admin	Rifiuta richiesta
PUT	/richieste/{id}/lavorazione	Admin	Avvia lavorazione
PUT	/richieste/{id}/terminata	Autenticato	Termina richiesta
GET	/missioni/list	Admin	Lista missioni
GET	/missioni/{id}	Admin	Dettaglio missione
GET	/missioni/{id}/squadra	Admin	Squadra della missione
GET	/missioni/{id}/mezzi	Admin	Mezzi della missione
GET	/missioni/{id}/materiali	Admin	Materiali della missione
PUT	/missioni/{id}/terminazione	Admin	Chiusura missione
POST	/missioni	Admin	Creazione missione
GET	/operatori/list	Admin	Lista operatori paginata
GET	/operatori/{id}	Autenticato	Dettaglio operatore
GET	/operatori/{id}/patenti	Autenticato	Patenti dell'operatore
GET	/operatori/{id}/abilita	Autenticato	Abilità dell'operatore
GET	/operatori/{id}/storico	Autenticato	Storico missioni dell'operatore
POST	/operatori	Admin	Creazione operatore
POST	/operatori/patenti	Autenticato	Associa patente
POST	/operatori/abilita	Autenticato	Associa abilità
GET	/mezzi/list	Admin	Lista mezzi paginata
GET	/mezzi/{id}	Admin	Dettaglio mezzo
POST	/mezzi	Admin	Creazione mezzo
GET	/materiali/list	Admin	Lista materiali paginata
GET	/materiali/{id}	Admin	Dettaglio materiale
POST	/materiali	Admin	Creazione materiale

Verbo	URL relativa	Accesso	Operazione
GET	/patenti/list	Autenticato	Lista patenti
GET	/abilita/list	Autenticato	Lista abilità
GET	/squadre/list	Autenticato	Lista squadre non operative

10.1 Autenticazione

Login

POST	/auth/login	Pubblico
------	-------------	----------

Autentica amministratori e operatori mediante email e password.

Parametro	Posizione	Obbl.	Tipo	Descrizione
email	form	sì	string	Indirizzo email dell'utente
password	form	sì	string	Password

Esempio di richiesta

```
email=roberto.mancini%40mail.com&password=roberto123
```

Risposta di successo - HTTP 200

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJyb2JlcjVlcnRvLi4u
```

HTTP	Significato
200	Token JWT nel corpo; cookie token e header Authorization.
401	Credenziali non valide.

Nota implementativa

Il ruolo viene cercato prima nella tabella amministratori e poi nella tabella operatori.

Logout

DELETE	/auth/logout	Autenticato
--------	--------------	-------------

Revoca il JWT corrente inserendolo nella blacklist e cancella il cookie.

Parametri/payload: nessuno.

Risposta di successo - HTTP 204

Nessun corpo di risposta.

HTTP	Significato
204	Logout completato, nessun corpo.
401	Token assente, scaduto, revocato o non valido.

Nota implementativa

Il token viene recuperato dal filtro e salvato in token_blacklist fino alla sua scadenza.

Refresh token

GET	/auth/refresh	Autenticato
-----	---------------	-------------

Emette un nuovo JWT usando email e ruolo del token valido corrente.

Parametri/payload: nessuno.

Risposta di successo - HTTP 200

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJyb2JlcnRvLi4u
```

HTTP	Significato
200	Nuovo token, cookie e header Authorization aggiornati.
401	Token corrente non valido.

Nota implementativa

La durata del nuovo token è di 15 minuti.

10.2 Convalida

Convalida richiesta

GET	/convalida	Pubblico
-----	------------	----------

Convalida la richiesta pendente associata al codice UUID inviato via email simulata.

Parametro	Posizione	Obbl.	Tipo	Descrizione
uuid	query	sì	string	Codice univoco della richiesta.

Risposta di successo - HTTP 200

```
{
  "message": "Richiesta attivata",
  "richiesta": {
    "id": 3,
    "nomePersona": "Anna Bianchi",
    "mailPersona": "anna@mail.com",
    "descrizione": "Allagamento della cantina dopo un temporale",
    "indirizzo": "Via Milano 22, L'Aquila",
    "createdAt": "2026-07-15T08:42:10",
    "workingAt": null,
    "closedAt": null,
    "fase": "attiva",
    "uuid": "8b0a6ald-0a24-42f6-a340-11e9e840a120"
  }
}
```

HTTP	Significato
200	Richiesta portata nello stato attiva.
400	Parametro UUID mancante.
404	Nessuna richiesta associata al codice.
409	Richiesta già convalidata o non più pendente.
500	Errore durante l'aggiornamento.

Nota implementativa

Il client pubblico usa la stessa pagina NuovaRichiesta.html per leggere il parametro uuid dalla URL.

10.3 Richieste

Lista richieste paginata

GET	/richieste/list	Admin
-----	-----------------	-------

Restituisce richieste non pendenti, con paginazione e filtro opzionale per fase.

Parametro	Posizione	Obbl.	Tipo	Descrizione
page	query	no	integer	Numero di pagina, a partire da 1.
size	query	no	integer	Elementi per pagina, da 1 a 100.
stato	query	no	string [attiva, in esecuzione, terminata, rifiutata]	Filtro: attiva, in esecuzione, terminata o rifiutata.

Risposta di successo - HTTP 200

```
{
  "content": [
    {
      "id": 3,
      "nomePersona": "Anna Bianchi",
      "mailPersona": "anna@mail.com",
      "descrizione": "Allagamento della cantina dopo un temporale",
      "indirizzo": "Via Milano 22, L'Aquila",
      "createdAt": "2026-07-15T08:42:10",
      "workingAt": null,
      "closedAt": null,
      "fase": "attiva",
      "uuid": "8b0a6a1d-0a24-42f6-a340-11e9e840a120"
    }
  ],
  "page": 1,
  "size": 10,
  "totalElements": 1,
  "totalPages": 1,
  "first": true,
  "last": true
}
```

HTTP	Significato
200	Pagina di richieste.
400	Paginazione o stato non validi.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.

Dettaglio richiesta

GET	/richieste/{id}	Admin
-----	-----------------	-------

Restituisce una richiesta tramite identificativo numerico.

Parametro	Posizione	Obbl.	Tipo	Descrizione
id	path	si	integer	Identificativo della richiesta.

Risposta di successo - HTTP 200

```
{
  "id": 3,
  "nomePersona": "Anna Bianchi",
  "mailPersona": "anna@mail.com",
  "descrizione": "Allagamento della cantina dopo un temporale",
  "indirizzo": "Via Milano 22, L'Aquila",
  "createdAt": "2026-07-15T08:42:10",
  "workingAt": null,
  "closedAt": null,
  "fase": "attiva",
  "uuid": "8b0a6a1d-0a24-42f6-a340-11e9e840a120"
}
```

```

"workingAt": null,
"closedAt": null,
"fase": "attiva",
"uuid": "8b0a6a1d-0a24-42f6-a340-11e9e840a120"
}

```

HTTP	Significato
200	Richiesta trovata.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.
404	Richiesta inesistente.

Inserimento richiesta

POST	/richieste	Pubblico
-------------	-------------------	-----------------

Registra una nuova richiesta nello stato pendente e genera un UUID di convalida.

Parametro	Posizione	Obbl.	Tipo	Descrizione
body	body	sì	RequestCreate	Payload JSON della richiesta.

Esempio di richiesta

```

{
  "nomePersona": "Anna Bianchi",
  "mailPersona": "anna@mail.com",
  "descrizione": "Allagamento della cantina dopo un temporale",
  "indirizzo": "Via Milano 22, L'Aquila"
}

```

Risposta di successo - HTTP 201

```

{
  "id": 3,
  "nomePersona": "Anna Bianchi",
  "mailPersona": "anna@mail.com",
  "descrizione": "Allagamento della cantina dopo un temporale",
  "indirizzo": "Via Milano 22, L'Aquila",
  "createdAt": "2026-07-15T08:42:10",
  "workingAt": null,
  "closedAt": null,
  "fase": "pendente",
  "uuid": "8b0a6a1d-0a24-42f6-a340-11e9e840a120"
}

```

HTTP	Significato
201	Richiesta creata.
400	Campi mancanti oppure stessa email usata negli ultimi due minuti.
500	Errore di persistenza.

Nota implementativa

L'invio email è simulato mediante scrittura su file locale.

Rifiuta richiesta

PUT	/richieste/{id}/rifiutata	Admin
------------	----------------------------------	--------------

Porta una richiesta attiva nello stato rifiutata.

Parametro	Posizione	Obbl.	Tipo	Descrizione
id	path	si	integer	Identificativo della richiesta.

Risposta di successo - HTTP 200

```
{
  "id": 3,
  "nomePersona": "Anna Bianchi",
  "mailPersona": "anna@mail.com",
  "descrizione": "Allagamento della cantina dopo un temporale",
  "indirizzo": "Via Milano 22, L'Aquila",
  "createdAt": "2026-07-15T08:42:10",
  "workingAt": null,
  "closedAt": null,
  "fase": "rifiutata",
  "uuid": "8b0a6a1d-0a24-42f6-a340-11e9e840a120"
}
```

HTTP	Significato
200	Richiesta aggiornata.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.
404	Richiesta inesistente.
409	La richiesta non è attiva.
500	Aggiornamento non riuscito.

Avvia lavorazione

PUT	/richieste/{id}/lavorazione	Admin
------------	------------------------------------	--------------

Porta una richiesta attiva nello stato in esecuzione.

Parametro	Posizione	Obbl.	Tipo	Descrizione
id	path	si	integer	Identificativo della richiesta.

Risposta di successo - HTTP 200

```
{
  "id": 3,
  "nomePersona": "Anna Bianchi",
  "mailPersona": "anna@mail.com",
  "descrizione": "Allagamento della cantina dopo un temporale",
  "indirizzo": "Via Milano 22, L'Aquila",
  "createdAt": "2026-07-15T08:42:10",
  "workingAt": null,
  "closedAt": null,
  "fase": "in esecuzione",
  "uuid": "8b0a6a1d-0a24-42f6-a340-11e9e840a120"
}
```

HTTP	Significato
200	Richiesta aggiornata.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.
404	Richiesta inesistente.
409	La richiesta non è attiva.
500	Aggiornamento non riuscito.

Termina richiesta

PUT	/richieste/{id}/terminata	Autenticato
------------	----------------------------------	--------------------

Porta una richiesta in esecuzione nello stato terminata.

Parametro	Posizione	Obbl.	Tipo	Descrizione
id	path	sì	integer	Identificativo della richiesta.

Risposta di successo - HTTP 200

```
{
  "id": 3,
  "nomePersona": "Anna Bianchi",
  "mailPersona": "anna@mail.com",
  "descrizione": "Allagamento della cantina dopo un temporale",
  "indirizzo": "Via Milano 22, L'Aquila",
  "createdAt": "2026-07-15T08:42:10",
  "workingAt": null,
  "closedAt": null,
  "fase": "terminata",
  "uuid": "8b0a6a1d-0a24-42f6-a340-11e9e840a120"
}
```

HTTP	Significato
200	Richiesta aggiornata.
401	Autenticazione mancante o non valida.
404	Richiesta inesistente.
409	La richiesta non è in esecuzione.
500	Aggiornamento non riuscito.

Nota implementativa

Nel codice corrente non è presente il controllo di ruolo admin.

10.4 Missioni

Lista missioni

GET	/missioni/list	Admin
------------	-----------------------	--------------

Restituisce tutte le missioni.

Parametri/payload: nessuno.

Risposta di successo - HTTP 200

```
[
  {
    "obiettivo": "Mettere in sicurezza l'abitazione e aspirare l'acqua",
    "posizione": "Via Milano 22, L'Aquila",
    "status": "attiva",
    "richiestaRif": 3,
    "squadraRif": 3,
    "missioneId": 8
  }
]
```

HTTP	Significato
200	Lista missioni.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.

Dettaglio missione

GET	/missioni/{id}	Admin
-----	----------------	-------

Restituisce i dati principali di una missione.

Parametro	Posizione	Obbl.	Tipo	Descrizione
id	path	sì	integer	Identificativo missione.

Risposta di successo - HTTP 200

```
{
  "obiettivo": "Mettere in sicurezza l'abitazione e aspirare l'acqua",
  "posizione": "Via Milano 22, L'Aquila",
  "status": "attiva",
  "richiestaRif": 3,
  "squadraRif": 3,
  "missioneId": 8
}
```

HTTP	Significato
200	Missione trovata.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.
404	Missione inesistente.

Squadra della missione

GET	/missioni/{id}/squadra	Admin
-----	------------------------	-------

Restituisce la squadra associati alla missione.

Parametro	Posizione	Obbl.	Tipo	Descrizione
id	path	sì	integer	Identificativo missione.

Risposta di successo - HTTP 200

```
{
  "nome": "Squadra Gamma",
  "squadraId": 3
}
```

HTTP	Significato
200	Risorsa associata restituita.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.
404	Missione o squadra inesistente.

Mezzi della missione

GET	/missioni/{id}/mezzi	Admin
-----	----------------------	-------

Restituisce l'elenco dei mezzi associati alla missione.

Parametro	Posizione	Obbl.	Tipo	Descrizione
id	path	sì	integer	Identificativo missione.

Risposta di successo - HTTP 200

```
[
  {
    "tipologia": "Ambulanza",
    "seriale": "MEZ09",
    "status": "libero",
    "id": 9
  }
]
```

HTTP	Significato
200	Risorsa associata restituita.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.
404	Missione o squadra inesistente.

Materiali della missione

GET	/missioni/{id}/materiali	Admin
------------	---------------------------------	--------------

Restituisce l'elenco dei materiali associati alla missione.

Parametro	Posizione	Obbl.	Tipo	Descrizione
id	path	si	integer	Identificativo missione.

Risposta di successo - HTTP 200

```
[
  {
    "tipologia": "Kit medico",
    "seriale": "MAT09",
    "status": "libero",
    "id": 9
  }
]
```

HTTP	Significato
200	Risorsa associata restituita.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.
404	Missione o squadra inesistente.

Chiusura missione

PUT	/missioni/{id}/terminazione	Admin
------------	------------------------------------	--------------

Termina una missione e libera squadra, mezzi e materiali; termina anche la richiesta associata.

Parametro	Posizione	Obbl.	Tipo	Descrizione
id	path	si	integer	Identificativo missione.

Risposta di successo - HTTP 200

```
{
  "obiettivo": "Mettere in sicurezza l'abitazione e aspirare l'acqua",
  "posizione": "Via Milano 22, L'Aquila",
  "status": "terminata",
  "richiestaRif": 3,
  "squadraRif": 3,
  "missioneId": 8
}
```

HTTP	Significato
200	Missione terminata.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.
404	Missione inesistente.
409	Nessuna squadra associata.

Nota implementativa

L'operazione coordina aggiornamenti su cinque DAO; nel codice corrente non è racchiusa in una transazione JDBC unica.

Creazione missione

POST	/missioni	Admin
------	-----------	-------

Crea una missione da una richiesta e associa squadra, mezzi e materiali.

Parametro	Posizione	Obbl.	Tipo	Descrizione
body	body	si	MissionCreateRequest	Payload JSON della richiesta.

Esempio di richiesta

```
{
  "richiestaId": 3,
  "obiettivo": "Mettere in sicurezza l'abitazione",
  "posizione": "Via Milano 22, L'Aquila",
  "squadraId": 3,
  "mezziIds": [
    2,
    5
  ],
  "materialiIds": [
    2,
    6
  ]
}
```

Risposta di successo - HTTP 201

```
{
  "obiettivo": "Mettere in sicurezza l'abitazione e aspirare l'acqua",
  "posizione": "Via Milano 22, L'Aquila",
  "status": "attiva",
  "richiestaRif": 3,
  "squadraRif": 3,
  "missioneId": 8
}
```

HTTP	Significato
201	Missione creata; header Location valorizzato.
400	Body o campi obbligatori mancanti.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.
404	Richiesta inesistente.
409	Esiste già una missione per la richiesta.
500	Creazione della missione non riuscita.

10.5 Operatori

Lista operatori paginata

GET	/operatori/list	Admin
-----	-----------------	-------

Restituisce gli operatori, filtrabili per stato libero o occupato.

Parametro	Posizione	Obbl.	Tipo	Descrizione
page	query	no	integer	Numero di pagina, a partire da 1.
size	query	no	integer	Elementi per pagina, da 1 a 100.
stato	query	no	string [libero, occupato]	Filtro: libero o occupato.

Risposta di successo - HTTP 200

```
{
  "content": [
    {
      "name": "Francesca",
      "surname": "Marini",
      "email": "francesca.marini@mail.com",
      "age": "1992-04-15",
      "id": 6,
      "status": "libero"
    }
  ],
  "page": 1,
  "size": 10,
  "totalElements": 1,
  "totalPages": 1,
  "first": true,
  "last": true
}
```

HTTP	Significato
200	Pagina operatori.
400	Paginazione o stato non validi.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.

Nota implementativa

Con stato=libero soddisfa la lista degli operatori attualmente disponibili richiesta dalla specifica.

Dettaglio operatore

GET	/operatori/{id}	Autenticato
-----	-----------------	-------------

Restituisce l'operatore e l'eventuale missione attiva.

Parametro	Posizione	Obbl.	Tipo	Descrizione
id	path	si	integer	Identificativo operatore.

Risposta di successo - HTTP 200

```
{
  "Operatore": {
    "name": "Francesca",
    "surname": "Marini",
    "email": "francesca.marini@mail.com",
    "age": "1992-04-15",
    "id": 6,
    "status": "libero"
  }
}
```

```

    },
    "MissioneAttiva": null
  }

```

HTTP	Significato
200	Dettaglio composito.
401	Autenticazione mancante o non valida.
404	Operatore inesistente.

Patenti dell'operatore

GET	/operatori/{id}/patenti	Autenticato
-----	-------------------------	-------------

Restituisce le patenti dell'operatore.

Parametro	Posizione	Obbl.	Tipo	Descrizione
id	path	si	integer	Identificativo operatore.

Risposta di successo - HTTP 200

```

[
  {
    "tipologia": "B",
    "id": 2
  }
]

```

HTTP	Significato
200	Lista restituita.
401	Autenticazione mancante o non valida.
404	Operatore inesistente.

Abilità dell'operatore

GET	/operatori/{id}/abilita	Autenticato
-----	-------------------------	-------------

Restituisce le abilità dell'operatore.

Parametro	Posizione	Obbl.	Tipo	Descrizione
id	path	si	integer	Identificativo operatore.

Risposta di successo - HTTP 200

```

[
  {
    "tipologia": "Primo Soccorso",
    "id": 1
  }
]

```

HTTP	Significato
200	Lista restituita.
401	Autenticazione mancante o non valida.
404	Operatore inesistente.

Storico missioni dell'operatore

GET	/operatori/{id}/storico	Autenticato
-----	-------------------------	-------------

Restituisce lo storico missioni dell'operatore.

Parametro	Posizione	Obbl.	Tipo	Descrizione
id	path	sì	integer	Identificativo operatore.

Risposta di successo - HTTP 200

```
[
  {
    "obiettivo": "Mettere in sicurezza l'abitazione e aspirare l'acqua",
    "posizione": "Via Milano 22, L'Aquila",
    "status": "terminata",
    "richiestaRif": 3,
    "squadraRif": 3,
    "missioneId": 8
  }
]
```

HTTP	Significato
200	Lista restituita.
401	Autenticazione mancante o non valida.
404	Operatore inesistente.

Creazione operatore

POST	/operatori	Admin
------	------------	-------

Registra un nuovo operatore con stato iniziale libero.

Parametro	Posizione	Obbl.	Tipo	Descrizione
body	body	sì	OperatorCreate	Payload JSON della richiesta.

Esempio di richiesta

```
{
  "name": "Francesca",
  "surname": "Marini",
  "email": "francesca.marini@mail.com",
  "age": "1992-04-15",
  "passkey": "francesca123"
}
```

Risposta di successo - HTTP 201

```
{
  "name": "Francesca",
  "surname": "Marini",
  "email": "francesca.marini@mail.com",
  "age": "1992-04-15",
  "id": 6,
  "status": "libero"
}
```

HTTP	Significato
201	Operatore creato; la passkey non viene serializzata nella risposta.
400	Campo obbligatorio mancante.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.

HTTP	Significato
500	Errore di persistenza.

Associa patente

POST	/operatori/patenti	Autenticato
------	--------------------	-------------

Associa una patente a un operatore tramite la tabella multi-a-molti.

Parametro	Posizione	Obbl.	Tipo	Descrizione
body	body	sì	PatenteOperatoreRequest	Payload JSON della richiesta.

Esempio di richiesta

```
{
  "operatoreRif": 6,
  "patenteRif": 2
}
```

Risposta di successo - HTTP 200

```
{
  "message": "Patente aggiunta all'operatore"
}
```

HTTP	Significato
200	Associazione creata.
400	Associazione non creata, ad esempio perché duplicata.
401	Autenticazione mancante o non valida.

Nota implementativa

Nel codice corrente non è presente il controllo di ruolo admin.

Associa abilità

POST	/operatori/abilita	Autenticato
------	--------------------	-------------

Associa un'abilità a un operatore tramite la tabella multi-a-molti.

Parametro	Posizione	Obbl.	Tipo	Descrizione
body	body	sì	AbilitaOperatoreRequest	Payload JSON della richiesta.

Esempio di richiesta

```
{
  "operatoreRif": 6,
  "abilitaRif": 1
}
```

Risposta di successo - HTTP 200

```
{
  "message": "Abilità aggiunta all'operatore"
}
```

HTTP	Significato
200	Associazione creata.
400	Associazione non creata, ad esempio perché duplicata.
401	Autenticazione mancante o non valida.

Nota implementativa

Nel codice corrente non è presente il controllo di ruolo admin.

10.6 Mezzi

Lista mezzi paginata

GET	/mezzi/list	Admin
-----	-------------	-------

Restituisce mezzi con filtro opzionale per stato libero o occupato.

Parametro	Posizione	Obbl.	Tipo	Descrizione
page	query	no	integer	Numero di pagina, a partire da 1.
size	query	no	integer	Elementi per pagina, da 1 a 100.
stato	query	no	string [libero, occupato]	Filtro: libero o occupato.

Risposta di successo - HTTP 200

```
{
  "content": [
    {
      "tipologia": "Ambulanza",
      "seriale": "MEZ09",
      "status": "libero",
      "id": 9
    }
  ],
  "page": 1,
  "size": 10,
  "totalElements": 1,
  "totalPages": 1,
  "first": true,
  "last": true
}
```

HTTP	Significato
200	Pagina di mezzi.
400	Paginazione o stato non validi.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.

Dettaglio mezzo

GET	/mezzi/{id}	Admin
-----	-------------	-------

Restituisce un mezzo tramite identificativo.

Parametro	Posizione	Obbl.	Tipo	Descrizione
id	path	sì	integer	Identificativo del mezzo.

Risposta di successo - HTTP 200

```
{
  "tipologia": "Ambulanza",
  "seriale": "MEZ09",
  "status": "libero",
  "id": 9
}
```

HTTP	Significato
200	Mezzo trovato.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.
404	Mezzo inesistente.

Creazione mezzo

POST	/mezzi	Admin
-------------	---------------	--------------

Registra un nuovo mezzo con stato iniziale libero.

Parametro	Posizione	Obbl.	Tipo	Descrizione
body	body	sì	MezzoCreate	Payload JSON della richiesta.

Esempio di richiesta

```
{
  "tipologia": "Ambulanza",
  "seriale": "MEZ09"
}
```

Risposta di successo - HTTP 201

```
{
  "tipologia": "Ambulanza",
  "seriale": "MEZ09",
  "status": "libero",
  "id": 9
}
```

HTTP	Significato
201	Mezzo creato.
400	Tipologia o seriale mancanti.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.
500	Errore di persistenza.

10.7 Materiali

Lista materiali paginata

GET	/materiali/list	Admin
------------	------------------------	--------------

Restituisce materiali con filtro opzionale per stato libero o occupato.

Parametro	Posizione	Obbl.	Tipo	Descrizione
page	query	no	integer	Numero di pagina, a partire da 1.

Parametro	Posizione	Obbl.	Tipo	Descrizione
size	query	no	integer	Elementi per pagina, da 1 a 100.
stato	query	no	string [libero, occupato]	Filtro: libero o occupato.

Risposta di successo - HTTP 200

```
{
  "content": [
    {
      "tipologia": "Kit medico",
      "seriale": "MAT09",
      "status": "libero",
      "id": 9
    }
  ],
  "page": 1,
  "size": 10,
  "totalElements": 1,
  "totalPages": 1,
  "first": true,
  "last": true
}
```

HTTP	Significato
200	Pagina di materiali.
400	Paginazione o stato non validi.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.

Dettaglio materiale

GET	/materiali/{id}	Admin
------------	------------------------	--------------

Restituisce un materiale tramite identificativo.

Parametro	Posizione	Obbl.	Tipo	Descrizione
id	path	sì	integer	Identificativo del materiale.

Risposta di successo - HTTP 200

```
{
  "tipologia": "Kit medico",
  "seriale": "MAT09",
  "status": "libero",
  "id": 9
}
```

HTTP	Significato
200	Materiale trovato.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.
404	Materiale inesistente.

Creazione materiale

POST	/materiali	Admin
-------------	-------------------	--------------

Registra un nuovo materiale con stato iniziale libero.

Parametro	Posizione	Obbl.	Tipo	Descrizione
body	body	sì	MaterialeCreate	Payload JSON della richiesta.

Esempio di richiesta

```
{
  "tipologia": "Kit medico",
  "seriale": "MAT09"
}
```

Risposta di successo - HTTP 201

```
{
  "tipologia": "Kit medico",
  "seriale": "MAT09",
  "status": "libero",
  "id": 9
}
```

HTTP	Significato
201	Materiale creato.
400	Tipologia o seriale mancanti.
401	Autenticazione mancante o non valida.
403	Ruolo diverso da admin.
500	Errore di persistenza.

10.8 Cataloghi

Lista patenti

GET	/patenti/list	Autenticato
-----	---------------	-------------

Restituisce tutte le tipologie di patente.

Parametri/payload: nessuno.

Risposta di successo - HTTP 200

```
[
  {
    "tipologia": "B",
    "id": 2
  }
]
```

HTTP	Significato
200	Lista patenti.
401	Autenticazione mancante o non valida.

Lista abilità

GET	/abilita/list	Autenticato
-----	---------------	-------------

Restituisce tutte le abilità disponibili.

Parametri/payload: nessuno.

Risposta di successo - HTTP 200

```
[
  {
```

```

    "tipologia": "Primo Soccorso",
    "id": 1
  }
]

```

HTTP	Significato
200	Lista abilità.
401	Autenticazione mancante o non valida.

10.9 Squadre

Lista squadre non operative

GET	/squadre/list	Autenticato
-----	---------------	-------------

Restituisce le squadre non impegnate in missioni attive e composte da operatori disponibili.

Parametri/payload: nessuno.

Risposta di successo - HTTP 200

```

[
  {
    "nome": "Squadra Gamma",
    "squadraId": 3
  }
]

```

HTTP	Significato
200	Lista squadre utilizzabili.
401	Autenticazione mancante o non valida.

11. Funzionalità extra

Extra	Descrizione
Refresh del JWT	GET /auth/refresh consente di rinnovare il token senza reinviare la password.
Blacklist persistente	Il logout rende inutilizzabile il token prima della scadenza naturale.
Paginazione estesa	Operatori, mezzi e materiali adottano la stessa struttura paginata delle richieste.
Filtri server-side	Richieste filtrate per fase e risorse filtrate per stato.
Swagger UI	Pagina dedicata con Try it out, filtro e persistenza dell'autorizzazione.
Sotto-risorse nidificate	Patenti, abilità, storico missioni, squadra, mezzi e materiali sono esposti con URL che evidenziano le relazioni.
Protezione della passkey in output	JsonProperty WRITE_ONLY impedisce la serializzazione della password dell'operatore.
Gestione automatica disponibilità	Creazione e chiusura missione aggiornano operatori, mezzi, materiali e richiesta.

12. Installazione, configurazione e collaudo

12.1 Prerequisiti

- JDK 21
- Apache Maven 3.9 o compatibile
- Apache Tomcat 10.1 o altro container Jakarta Servlet 6
- MySQL 8.x
- Browser moderno con Fetch API

12.2 Database

4. Eseguire Documentazione/Database/Esame.sql su MySQL.
5. Verificare che il database soccorso_web sia raggiungibile su localhost:3306.
6. Aggiornare DBConfig o, preferibilmente, esternalizzare URL, utente e password.

Configurazione presente nel sorgente:

```
URL = jdbc:mysql://localhost:3306/soccorso_web?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
USER = root
PASSWORD = toor
```

12.3 Build e deploy

```
mvn clean package
# copiare target/soccorso_Web_SWA-0.0.1-SNAPSHOT.war nella directory webapps di Tomcat
```

12.4 URL principali

Risorsa	URL
Client pubblico	http://localhost:8080/soccorso_Web_SWA/viste/NuovaRichiesta.html
Login	http://localhost:8080/soccorso_Web_SWA/viste/Login.html
Base REST	http://localhost:8080/soccorso_Web_SWA/rest
OpenAPI JSON	http://localhost:8080/soccorso_Web_SWA/rest/openapi.json
Swagger UI	http://localhost:8080/soccorso_Web_SWA/swagger-ui.html

12.5 Sequenza minima di collaudo

7. POST /auth/login con un amministratore di esempio e conservare cookie/token.
8. GET /richieste/list?page=1&size=10 per verificare autenticazione e paginazione.
9. POST /richieste senza autenticazione, poi GET /convalida?uuid=<codice>.
10. GET /operatori/list?stato=libero e GET /squadre/list per selezionare le risorse.
11. POST /missioni e verificare che richiesta e risorse passino allo stato operativo.
12. PUT /missioni/{id}/terminazione e verificare il ripristino dello stato libero.
13. DELETE /auth/logout e riprovare un endpoint protetto: la risposta attesa è 401.

Appendice A - Schemi JSON

Gli esempi seguenti riportano la struttura completa dei principali dati scambiati. I campi null sono ammessi per timestamp non ancora valorizzati e per la missione attiva assente.

Request

```
{
  "id": 3,
  "nomePersona": "Anna Bianchi",
  "mailPersona": "anna@mail.com",
  "descrizione": "Allagamento della cantina dopo un temporale",
  "indirizzo": "Via Milano 22, L'Aquila",
  "createdAt": "2026-07-15T08:42:10",
  "workingAt": null,
  "closedAt": null,
  "fase": "attiva",
  "uuid": "8b0a6a1d-0a24-42f6-a340-11e9e840a120"
}
```

Missione

```
{
  "obiettivo": "Mettere in sicurezza l'abitazione e aspirare l'acqua",
  "posizione": "Via Milano 22, L'Aquila",
  "status": "attiva",
  "richiestaRif": 3,
  "squadraRif": 3,
  "missioneId": 8
}
```

Operator

```
{
  "name": "Francesca",
  "surname": "Marini",
  "email": "francesca.marini@mail.com",
  "age": "1992-04-15",
  "id": 6,
  "status": "libero"
}
```

Dettaglio operatore

```
{
  "Operatore": {
    "name": "Francesca",
    "surname": "Marini",
    "email": "francesca.marini@mail.com",
    "age": "1992-04-15",
    "id": 6,
    "status": "libero"
  },
  "MissioneAttiva": null
}
```

MissionCreateRequest

```
{
  "richiestaId": 3,
  "obiettivo": "Mettere in sicurezza l'abitazione",
  "posizione": "Via Milano 22, L'Aquila",
  "squadraId": 3,
  "mezziIds": [
    2,
  ]
}
```

```

    5
  ],
  "materialiIds": [
    2,
    6
  ]
}

```

Mezzo

```

{
  "tipologia": "Ambulanza",
  "seriale": "MEZ09",
  "status": "libero",
  "id": 9
}

```

Materiale

```

{
  "tipologia": "Kit medico",
  "seriale": "MAT09",
  "status": "libero",
  "id": 9
}

```

Squadra

```

{
  "nome": "Squadra Gamma",
  "squadraId": 3
}

```

PatenteOperatoreRequest

```

{
  "operatoreRif": 6,
  "patenteRif": 2
}

```

AbilitaOperatoreRequest

```

{
  "operatoreRif": 6,
  "abilitaRif": 1
}

```

ErrorResponse

```

{
  "message": "Richiesta non trovata"
}

```

SuccessResponse

```

{
  "message": "Patente aggiunta all'operatore"
}

```